

Objetivo e ligação com o mundo real

Objetivo

O objetivo desta aula é entender **como as linguagens avaliam expressões e controlam o fluxo de execução**, para que o seu código faça exatamente o que parece fazer, **sem surpresas** de precedência, conversão ou desvio.

Para o engenheiro de software, isso significa:

- ler código alheio sem ser enganado pela indentação ou pela precedência
- evitar conversões implícitas perigosas (== do JS e do PHP)
- escolher a estrutura de controle mais legível para cada caso

Quando o fluxo sai do controle

2014 · "goto fail" (Apple)

Uma linha `goto fail`; duplicada pulava a verificação de assinatura: iOS e macOS aceitavam certificados falsos.

2008 · Tocadores Zune (Microsoft)

No último dia de um ano bissexto, um laço `while` nunca terminava, e todos os aparelhos travaram.

PHP · comparação com ==

`"0e4..." == "0e8..."` é verdadeiro: logins que comparavam hashes com == aceitavam senhas erradas.

Sebesta, 11ª ed. - Caps. 7 e 8 - p. 299-361

O slide diz que a aula quer ensinar como as linguagens de programação decidem a ordem em que o código roda, pra que ele sempre faça o que parece fazer, sem surpresas.

Isso ajuda a ler o código dos outros sem se confundir, evitar que a linguagem converta coisas sozinha de um jeito perigoso e escolher a forma mais clara de organizar o programa.

Pra mostrar que isso importa, ele dá três exemplos de erros reais: uma linha repetida na Apple que deixou aceitar certificados falsos, um erro num aparelho da Microsoft que travou todos no último dia de um ano bissexto, e uma comparação no PHP que deixava entrar com a senha errada.

Roteiro da aula (80 min)

1

0-10 min

Quiz de abertura: preveja a saída

2

10-30 min

Expressões: precedência, efeitos colaterais, sobrecarga

3

30-45 min

Conversões, expressões booleanas e curto-circuito

4

45-55 min

Sentenças de atribuição

5

55-70 min

Seleção, iteração, goto e comandos protegidos

6

70-80 min

Exercício: encontre o bug

Tipo de aula: exploratória. A turma prevê a saída antes de executar.

Exemplos: pasta `exemplos/`, na ordem dos slides. Todos rodam no navegador em onecompiler.com.

Esse slide mostra o roteiro da aula de 80 minutos, dividido em seis partes: começa com um quiz de abertura onde a turma tenta prever a saída do código, depois passa por expressões, conversões e curto-circuito, atribuições, estruturas de controle (seleção, repetição e desvios) e termina com um exercício de encontrar o bug.

A aula é exploratória, ou seja, a turma tenta adivinhar o resultado antes de rodar. Os exemplos ficam numa pasta, seguem a ordem dos slides e rodam direto no navegador pelo onecompiler.com.

Quiz de abertura: o que cada linha imprime?

Anote o seu palpite para cada trecho **antes** de executar. Vamos conferir ao longo da aula.

1 · Python

```
print(-2 ** 2, 2 ** 3 ** 2)
```

2 · Java

```
System.out.println(1 + 2 + "3" + 4 + 5);
```

3 · JavaScript

```
console.log("5" - 2, "5" + 2);
```

4 · C

```
printf("%d\n", 3 > 2 > 1);
```

5 · Python

```
x = 0  
print(x or "padrão", 3 > 2 > 1)
```

6 · JavaScript

```
console.log(0 == "", "" == "0", 0 == "0");
```

As respostas aparecem ao longo da aula - Sebesta Cap. 7

1) Python: -4 512

(** vem antes do menos unário e associa à direita)

2) Java: 3345

(+ associa à esquerda: 1+2=3, depois vira concatenação)

3) JavaScript: 3 52

(- converte para número, + concatena)

4) C: 0

(3>2 vale 1, e 1>1 é falso)

5) Python: padrão True

(0 é falso, então o or devolve o 2º operando; 3>2>1 é encadeado)

6) JavaScript: true false true

(== converte tipos e não é transitivo)

Expressões aritméticas: questões de projeto

- Quais são as regras de **precedência**?
- Quais são as regras de **associatividade**?
- Em que **ordem os operandos** são avaliados?
- Há restrições a **efeitos colaterais**?
- O usuário pode **sobrecarregar** operadores?
- Que **mistura de tipos** é permitida?

Lisp dispensa a precedência (p. 306)

(+ a (* b c)): notação prefixa, e os parênteses dizem a ordem.

$a + b * c$ com $a=3, b=4, c=5$

da esquerda para a direita

$(3 + 4) * 5 = 35$

da direita para a esquerda

$3 + (4 * 5) = 23$

com precedência (* antes de +)

$3 + 4 * 5 = 23$

APL: sempre da direita para a esquerda (p. 304)

$3 * 4 + 5 = 27$

Sebesta, 11ª ed. - §7.2 e §7.2.1 - p. 300-305

Imagina que você escreve uma conta, tipo $3 + 4 \times 5$, e cada linguagem de programação resolve de um jeito. Quem cria a linguagem precisa decidir várias coisas: o que vem primeiro, como desempatar quando duas operações têm o mesmo peso, em que ordem os valores são lidos e até se o programador pode mudar o significado dos símbolos.

O slide mostra que a mesma conta pode dar resultados diferentes. Se resolver da esquerda pra direita, dá 35. Se for da direita pra esquerda, dá 23. Na matemática normal, em que a multiplicação vem antes, também dá 23, e é isso que a maioria das linguagens faz. Já o APL resolve sempre da direita pra esquerda e chega em 27.

Tem ainda o Lisp, que nem se preocupa com essa briga: ele usa parênteses pra mostrar a ordem, então não existe dúvida.

No fim, a mensagem é que a ordem das contas não é óbvia, e quem programa precisa saber qual regra a linguagem está usando.

Precedência e associatividade na prática

Expressão	Python	JavaScript	Java
<code>-2 ** 2</code>	-4	erro de sintaxe	(não há <code>**</code>)
<code>2 ** 3 ** 2</code>	512	512	—
<code>1 + 2 + "3"</code>	<code>TypeError</code>	<code>"33"</code>	<code>"33"</code>
<code>-7 / 2</code> (inteiros)	-4 (com <code>//</code>)	-3.5	-3
<code>-7 % 2</code>	1	-1	-1

Associatividade (p. 303)

Esquerda: `*` `/` `%` `+` `-`
Direita: `**` (Python, JS, Ruby, Fortran),
`++`, `--`, menos unário
Visual Basic: `^` à esquerda!

Excel

`= -2^2` dá 4: lá o menos unário vence o `^`.

Na dúvida, **use parênteses** (p. 305): eles deixam a intenção explícita para o compilador e para o próximo leitor.

Sebesta, 11ª ed. - §7.2.1.1-7.2.1.3 - p. 301-305 - exemplos/01_precedencia

Esse slide mostra que a mesma conta pode dar resultados diferentes dependendo da linguagem. Python, JavaScript e Java tratam potência, soma de número com texto, divisão de negativos e resto de formas distintas, então não dá pra assumir que tudo funciona igual em todo lugar.

Ele também lembra que, quando duas operações têm o mesmo peso, cada linguagem decide se resolve da esquerda ou da direita (a potência, por exemplo, costuma ir da direita), e que até o Excel tem suas manias, como o menos na frente ganhar da potência.

A dica final é simples: na dúvida, use parênteses, porque deixam a intenção clara tanto pro computador quanto pra quem for ler seu código depois.

Ordem dos operandos e efeitos colaterais

Exemplo do livro (p. 307)

```
int a = 5;

int fun1(void) {
    a = 17;      /* efeito colateral */
    return 3;
}

int main(void) {
    a = a + fun1(); /* 8 ou 20? */
    printf("a = %d\n", a);
}
```

8

Java

a ordem é garantida: da esquerda para a direita (p. 308)

20

C (gcc 14)

a ordem não é definida pelo padrão

20

Go

a especificação não define a leitura de a em relação à chamada

Transparência referencial (p. 308): sem efeitos colaterais, a ordem deixa de importar, e $f(x) + f(x)$ pode virar $2 * f(x)$.

Sebesta, 11ª ed. - §7.2.2 - p. 307-309 - exemplos/02_efeitos_colaterais

Esse slide mostra que, quando uma função muda o valor de uma variável enquanto a conta está sendo feita, o resultado depende da ordem em que a linguagem lê as coisas. No exemplo, a conta pode dar 8 ou 20: em Java o resultado é sempre 8, porque a ordem é garantida da esquerda para a direita, mas em C e em Go dá 20, porque a linguagem não define essa ordem.

A lição é que, se a função não mexe em nada fora dela (sem efeitos colaterais), a ordem deixa de importar e o resultado é sempre previsível. Isso se chama transparência referencial.

Operadores sobrecarregados

- O mesmo símbolo com **vários significados**: + soma inteiros, reais e concatena (Java)
- Risco (p. 309): & em C++ é E bit a bit e "endereço de"
- Definida pelo usuário (p. 404): C++, C#, Python, Rust, Kotlin. Java não permite

Java

```
System.out.println('a' + 1);      // 98
System.out.println("" + 'a' + 1); // a1
x.add(y).multiply(tres);          // BigDecimal: sem +
```

Python · sobrecarga com métodos especiais

```
class Vetor:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, outro):      # v1 + v2
        return Vetor(self.x + outro.x,
                       self.y + outro.y)

    def __mul__(self, k):          # v * 3
        return Vetor(self.x * k, self.y * k)

v = Vetor(1, 2) + Vetor(3, 4)    # Vetor(4, 6)
print(v * 3)                     # Vetor(12, 18)
```

Sebesta, 11ª ed. - §7.3 - p. 309-310 e §9.11 - p. 404 - exemplos/03_sobrecarga

Esse slide fala de sobrecarga de operadores, que é quando o mesmo símbolo passa a ter significados diferentes dependendo do contexto. O "+", por exemplo, soma números, mas também junta textos em Java. O risco é a confusão: o "&" em C++ pode significar duas coisas bem distintas.

Algumas linguagens, como C++, C#, Python, Rust e Kotlin, deixam o próprio programador definir o que cada símbolo faz com os seus tipos, como somar dois vetores com "+". Java não permite isso, então lá você precisa usar métodos, como no caso do BigDecimal.

Conversões de tipos: alargamento, estreitamento e coerção

Java · coerção e cast

```
int grande = 123456789;
float f = grande;      // alargamento implícito
System.out.println((int) f); // 123456792 !

double d = 3.99;
System.out.println((int) d); // 3 (cast)
System.out.println((byte) 200); // -56

byte b = 10, c = 20;
// byte s = b + c; // erro: b + c é int (p. 312)

System.out.println(1 / 2); // 0
System.out.println(1 / 2.0); // 0.5
```

Linguagem	int → real (implícito)	int32 + int64
C	sim	coerção
Java	sim (alargamento)	coerção para long
Python	sim	(um só tipo int)
Go	não: float64(i)	erro de compilação
Rust	não: i as f64	erro de compilação
ML, F# (p. 312)	não	erro de compilação

Coerção esconde erros (p. 312): em `d = b * a`; com a digitado por engano no lugar de c, o Java converte e compila.

Sebesta, 11ª ed. - §7.4 - p. 310-313 - exemplos/04_conversoes

Esse slide fala de conversão de tipos, ou seja, quando a linguagem transforma um tipo de número em outro. Quando o destino é maior, ela faz isso sozinha (alargamento), mas pode perder precisão, como um número grande que muda de valor ao virar decimal. Quando o destino é menor, o programador precisa pedir de forma explícita (estreitamento), e aí o valor pode sair truncado ou até bem diferente do esperado.

Cada linguagem lida com isso de um jeito. C, Java e Python fazem muita conversão automática, enquanto Go, Rust, ML e F# exigem que você peça tudo explicitamente e dão erro se misturar tipos.

O alerta é que a conversão automática pode esconder erros: se você digitar uma variável errada, o Java converte e compila sem reclamar, e o bug passa batido.

Coerção implícita: JavaScript e PHP

JavaScript

```
"5" - 2           // 3
"5" + 2           // "52"
"5" * "2"         // 10
true + 1          // 2
[] + {}           // "[object Object]"
null + 1          // 1
undefined + 1     // NaN
"b" + "a" + +"a" + "a" // "baNaNaN"

0 == ""          // true
"" == "0"        // false
0 == "0"         // true  (não é transitivo!)
0 === "0"        // false (=== não converte)
```

PHP - "type juggling"

```
var_dump("0e123" == "0e456"); // true !
var_dump("0e123" === "0e456"); // false
var_dump("1" == "01"); // true

md5("240610708") // "0e462097431906...."
md5("QNKCDZO") // "0e830400451993...."
// == entre os dois hashes: true
```

Regra prática

Em JavaScript e PHP, compare com === e !==, que não fazem coerção.

Sebesta, 11ª ed. - §7.4.1 Coerção em expressões - p. 311-312 - exemplos/04_conversoes

Esse slide mostra como JavaScript e PHP tentam "adivinhar" o que você quis dizer quando mistura tipos diferentes, e acabam gerando resultados esquisitos. No JavaScript, "5" - 2 dá 3, mas "5" + 2 vira o texto "52", e até somar listas ou valores vazios produz respostas sem sentido. A comparação também engana: 0 é igual a "" e 0 é igual a "0", mas "" não é igual a "0", o que quebra a lógica.

No PHP acontece algo parecido: dois textos diferentes que parecem números em notação científica são considerados iguais, e isso fazia hashes totalmente distintos passarem como se fossem o mesmo.

A regra prática é usar sempre a comparação estrita (=== e !==), que não faz conversão automática e compara de verdade o valor e o tipo.

Expressões relacionais e booleanas

C · booleano é inteiro (p. 315)

```
int a = 3, b = 2, c = 1;
printf("%d\n", a > b > c);    // 0 !
// (3 > 2) vale 1, e 1 > 1 é falso

printf("%d\n", (a > b) && (b > c));    // 1

if (-1) printf("verdadeiro\n");    // != 0
```

Python · comparações encadeadas

```
a, b, c = 3, 2, 1
print(a > b > c)    # True: (a > b) and (b > c)

# contam como FALSO em um if:
# 0    0.0    ""    []    {}    None
# contam como VERDADEIRO:
# "0"    [0]    " "
```

Linguagem	O que o if aceita
C, Python, JavaScript	qualquer valor (0, vazio e nulo contam como falso)
Java, C#, Go, Rust	somente booleano: if (n) com n inteiro nem compila

Sebesta, 11ª ed. - §7.5 - p. 313-315 - exemplos/05_booleanas

Esse slide mostra que cada linguagem tem sua própria ideia do que é "verdadeiro" e "falso". Em C, o verdadeiro e o falso são na verdade números (1 e 0), então uma comparação encadeada como "3 > 2 > 1" dá falso: primeiro ele resolve 3 > 2, que vira 1, e depois compara 1 > 1. Já no Python, essa mesma comparação funciona como a gente espera, dando verdadeiro, porque ele entende como "3 > 2 e 2 > 1".

Outra diferença está no que o "if" aceita. Em C, Python e JavaScript, qualquer valor serve, e coisas como zero, texto vazio ou nulo contam como falso. Já em Java, C#, Go e Rust, o "if" só aceita um valor realmente booleano, e se você colocar um número, o código nem compila.

Avaliação em curto-circuito

Java - && faz curto-circuito, & não

```
// busca do livro (p. 316)
while (i < lista.length && lista[i] != chave)
    i++;           // && evita lista[lista.length]

String s = null;
s != null && s.length() > 0 // false
s != null & s.length() > 0 // NullPointerException
```

JavaScript - ?. e ??

```
const u = { nome: "Ana", endereco: null };
u.endereco?.cidade // undefined, sem erro

const qtd = 0;
qtd || 10           // 10 (o 0 se perdeu!)
qtd ?? 10           // 0  (só troca null/undefined)
```

Cuidado com efeitos colaterais (p. 316)

Em $(a > b) || (b++ / 3)$, o $b++$ só acontece quando $a \leq b$. Em Python, 0 or "padrão" devolve "padrão"; and e or devolvem um dos operandos.

Sebesta, 11ª ed. - §7.6 - p. 315-317 - exemplos/05_booleanas

Esse slide fala de curto-circuito, que é quando a linguagem para de avaliar uma condição assim que já sabe o resultado, sem olhar o resto. Isso é útil pra evitar erros: em Java, o "&&" confere primeiro se o valor existe antes de usá-lo, então nada quebra, enquanto o "&" avalia os dois lados sempre e acaba dando erro quando o valor é nulo.

No JavaScript, existem atalhos pra lidar com valores vazios sem travar. O "?. " acessa algo só se existir, e o "?? " usa um valor padrão apenas quando o original é nulo ou indefinido, diferente do "||", que também descartaria um zero que era válido.

O cuidado é com efeitos colaterais: como a linguagem pode pular parte da conta, qualquer coisa que estivesse nessa parte pode simplesmente não acontecer. E em Python, "and" e "or" devolvem um dos próprios valores, não só verdadeiro ou falso.

Sentenças de atribuição

C · a atribuição é uma expressão (p. 319-320)

```
sum = count = 0;           // múltiplos alvos
a = b + (c = d / b) - 1;    // exemplo do livro

while ((ch = getchar()) != EOF) { ... }

if (x = y) { ... } // compila! atribui e testa y
```

Python · atribuição múltipla (p. 320)

```
a, b = b, a                # troca sem temporária
if (n := len(dados)) > 5: ... # morsa (Python 3.8+)
```

Linguagem	Atribuição é expressão?
C, C++, JavaScript	sim: if (x = y) compila
Java, C#	sim, mas o if exige booleano
Python	não, exceto o operador morsa :=
Go	não: nem x++ é expressão
Rust	vale (): if x = 5 não compila

Linguagens funcionais (p. 321)

val (ML) e let (F#, Rust) dão nome a um valor, que não muda.

Sebesta, 11ª ed. · §7.7 · p. 317-321 · exemplos/06_atribuicao

Esse slide fala de atribuição, que é quando você guarda um valor numa variável. Em algumas linguagens, como C, C++ e JavaScript, a atribuição também vale como uma conta que devolve um resultado, então dá pra encadear várias de uma vez ou até colocar dentro de um "if". O perigo é escrever "=" no lugar de "==" sem querer: o código compila, atribui o valor e ainda testa, escondendo o erro.

Outras linguagens são mais rígidas. Em Java e C#, a atribuição também devolve valor, mas o "if" exige um booleano de verdade, o que barra esse tipo de engano. No Python ela normalmente não é uma conta (a exceção é o operador morsa, ":="), mas ele permite atribuição múltipla, como trocar dois valores numa linha só. Go e Rust são ainda mais restritos.

Nas linguagens funcionais, o conceito muda: você dá um nome a um valor, e esse valor não muda depois.

Atribuição de modo misto

Java · só alargamento (p. 322)

```
double d = 3;           // ok: alargamento
// int i = 3.7;         // erro: estreitamento
int i = (int) 3.7;       // 3 (cast explícito)
long l = i;              // ok

i += 3.7;                // COMPILA: cast escondido
System.out.println(i);   // 6
```

C · coerção em qualquer direção

```
int i = 3.7;             // 3, em silêncio
char c = 300;            // 44 (só um aviso)
unsigned int u = -1;      // 4294967295
```

Armadilha clássica

`double taxa = acertos / total * 100;` com inteiros dá 0.0: a divisão inteira acontece antes do alargamento para double.

Sebesta, 11ª ed. - §7.8 - p. 321-322 - exemplos/06_atribuicao

Esse slide fala de atribuição de modo misto, que é quando você guarda um valor de um tipo numa variável de outro tipo. O Java só aceita isso sozinho quando o destino é maior (de inteiro pra decimal, por exemplo). Se for o contrário, ele dá erro e exige que você peça a conversão de forma explícita. Já o C aceita em qualquer direção e em silêncio: um decimal vira inteiro cortando a parte depois da vírgula, e um número grande demais ou negativo vira outro valor completamente diferente, às vezes com um simples aviso.

Mesmo no Java existe uma brecha: o "+=" faz a conversão escondida, então somar um decimal a um inteiro compila sem reclamar e o resultado sai truncado.

A armadilha clássica é calcular uma porcentagem com números inteiros. A divisão inteira acontece antes da conversão pra decimal, então o resultado dá 0.0, e o erro passa despercebido.

Estruturas de controle: quantas são necessárias?

Sequência

uma sentença após a outra

```
a = 1;  
b = a + 2;
```

Seleção

escolher um caminho

```
if (x > 0) ...  
else ...
```

Iteração

repetir enquanto uma condição vale

```
while (x > 0)  
    x--;
```

Böhm e Jacopini (1966): sequência, seleção e laço lógico bastam para qualquer algoritmo. O goto não é necessário.

Por que então tantas outras (for, switch, do, foreach)? **Facilidade de escrita.** O limite é a **legibilidade**: quanto maior a linguagem, menos do código alheio você reconhece (p. 329).

Sebesta, 11ª ed. - §8.1 - p. 328-330

Esse slide pergunta quantas estruturas de controle uma linguagem realmente precisa. A resposta é que bastam três: sequência (uma instrução depois da outra), seleção (escolher um caminho, como o "if") e iteração (repetir enquanto uma condição for verdadeira, como o "while"). Em 1966, Böhm e Jacopini provaram que só com isso dá pra escrever qualquer algoritmo, e o "goto" não é necessário.

Então por que existem tantas outras, como "for", "switch" e "foreach"? Pra facilitar a escrita. O problema é que, quanto mais estruturas uma linguagem tem, menos código dos outros você consegue reconhecer de primeira, e a legibilidade é o limite.

Seleção de dois caminhos: o "else pendente"

C · a endentação engana (p. 331)

```
int sum = 0, count = 5, result = -1;  
if (sum == 0)  
    if (count == 0)  
        result = 0;  
else // de qual if é este else?  
    result = 1;  
printf("%d\n", result); // 1
```

Regra de Java, C, C++ e C#

O else pertence ao if **mais próximo** que ainda não tem else. A endentação é ignorada (exceto em Python e F#).

Chaves obrigatórias eliminam o problema: Perl, Go e Rust (p. 332).

Seletores que devolvem valor

```
let y = if x > 0 { x } else { 2 * x };  
y = x if x > 0 else 2 * x
```

// Rust: if é uma expressão (como em F#, p. 334)
Python: expressão condicional (p. 306)

Sebesta, 11ª ed. - §8.2.1 - p. 330-334 - exemplos/07_selecao

Esse slide fala do problema do "else pendente", que aparece quando há um "if" dentro de

outro e só um "else": a dúvida é a qual dos dois ele pertence. No exemplo em C, a endentação sugere que o "else" é do primeiro "if", mas a linguagem ignora a endentação e o liga ao "if" mais próximo que ainda não tem "else". Resultado: o código faz outra coisa do que parece. Isso vale pra Java, C, C++ e C#, enquanto Python e F# usam a endentação de verdade.

Algumas linguagens, como Perl, Go e Rust, resolvem o problema exigindo chaves sempre, então não há ambiguidade.

O slide também mostra que, em linguagens como Rust, F# e Python, o "if" pode devolver um valor, funcionando como uma expressão que escolhe entre duas opções e já guarda o resultado numa variável.

Seleção múltipla: do switch de C ao match

C · sem break, o controle "cai" (p. 335)

```
switch (index) {
  case 1:
  case 3: odd += 1;
  case 2:
  case 4: even += 1;
  default: printf("Error\n");
}
// index = 1: odd, even E erro!
```

Java 14+ · switch como expressão

```
String tipo = switch (dia) {
  case SAB, DOM -> "fim de semana";
  case SEG, TER, QUA,
    QUI, SEX -> "dia útil";
}; // sem break, sem cair;
// exaustivo para enum
```

Rust · match exaustivo

```
match n {
  i32::MIN..=-1 => "negativo",
  0 => "zero",
  1..=9 => "um dígito",
  _ => "vários dígitos",
}
// sem o _ : não compila
```

Go: cada caso termina sozinho, e cair exige `fallthrough`. C#: cair implicitamente é proibido (p. 336). Python 3.10: `match` com padrões e guardas.

Sebesta, 11ª ed. - §8.2.2 - p. 334-341 - exemplos/07_selecao

Esse slide fala de seleção múltipla, que é quando você escolhe entre vários caminhos de uma vez, e mostra como isso evoluiu. No "switch" do C, se você esquece o "break", o controle "cai" para o caso seguinte e executa o que não devia, o que causa erros fáceis de passar batido.

O Java mais novo corrigiu isso: o "switch" virou uma expressão que devolve um valor, sem "break" e sem cair de um caso no outro. Já o "match" do Rust vai além e exige que você cubra todas as possibilidades, e se esquecer alguma, o código nem compila.

Outras linguagens também resolveram do seu jeito: em Go cada caso termina sozinho, em C# cair de um caso no outro é proibido, e o Python 3.10 ganhou um "match" com padrões e condições extras.

Laços controlados por contador

C · o for é um laço lógico disfarçado

```
for (int i = 0; i < 10; i++) {  
    printf("%d ", i);  
    i += 2;           // altera a contagem  
}  
                        // 0 3 6 9  
  
for (; i <= 100; ) // expressões opcionais  
    soma += i++;
```

Python · o for percorre um iterável

```
for i in range(0, 10):  
    print(i, end=" ")  
    i += 2           # NÃO muda nada  
                        # 0 1 2 3 4 5 6 7 8 9  
  
for i in range(10, 0, -3): # 10 7 4 1  
    ...
```

Questões de projeto (p. 342)

Qual é o tipo e o escopo da variável? Posso alterá-la no corpo? Os parâmetros são avaliados uma vez ou a cada volta?

Sebesta, 11ª ed. - §8.3.1 - p. 342-345 - exemplos/08_iteracao

Esse slide fala de laços controlados por contador, aqueles que repetem algo um número determinado de vezes. No C, o "for" é na verdade um laço lógico disfarçado: você pode mexer na variável de contagem dentro do corpo e isso muda o rumo da repetição, como pular números. Já no Python, o "for" percorre uma sequência pronta, então alterar a variável dentro do laço não muda nada, porque a próxima volta pega o valor seguinte da sequência de qualquer jeito. Dá também pra contar de trás pra frente ou de três em três.

No fim, o slide levanta perguntas que quem projeta uma linguagem precisa responder: qual é o tipo e o alcance da variável de contagem, se dá pra alterá-la dentro do laço e se os limites são calculados uma vez só ou a cada volta.

Laços lógicos, saídas e iteradores

Java · break rotulado (p. 348)

```
externo:
for (int i = 0; i < m.length; i++)
  for (int j = 0; j < m[i].length; j++)
    if (m[i][j] < 0) {
      System.out.println(i + ", " + j);
      break externo; // sai dos DOIS laços
    }
```

JavaScript · for...in × for...of

```
const notas = [10, 20, 30];
let total = 0;
for (const i in notas) total += i;
console.log(total); // "0012": chaves!

total = 0;
for (const v of notas) total += v;
console.log(total); // 60: valores
```

Zune (2008) · o laço lógico sem progresso

```
while (dias > 365) {
  if (bissexto(ano)) {
    if (dias > 366) { dias -= 366; ano++; }
    // dias == 366: nada muda, laço infinito!
  } else { dias -= 365; ano++; }
}
```

Pós-teste (do-while) executa ao menos uma vez, e isso é fácil de esquecer (p. 347).

Sebesta, 11ª ed. · §8.3.2-8.3.4 · p. 345-352 · exemplos/08_iteracao

Esse slide fala de laços que repetem até uma condição deixar de valer, e das formas de sair deles. Em Java, o "break" rotulado permite sair de dois laços aninhados de uma vez só, em vez de só do mais interno.

No JavaScript, há uma pegadinha: o "for...in" percorre as chaves (posições) de uma lista, não os valores, então somar as notas desse jeito junta textos e dá um resultado absurdo. Já o "for...of" percorre os valores de verdade e dá a soma certa.

O slide também volta ao bug do Zune: num dia específico, a condição do laço nunca mudava nada, então ele ficava preso para sempre. É o clássico laço sem progresso.

Por fim, lembra que o "do-while" sempre executa ao menos uma vez, porque testa a condição só no final, e isso é fácil de esquecer.

Desvio incondicional: o goto

"goto fail" (Apple, 2014) - versão simplificada

```
int valida_certificado(void) {  
    int err;  
    if ((err = verifica_hash()) != 0)  
        goto fail;  
    goto fail;           // duplicada: desvia SEMPRE  
    if ((err = verifica_assinatura()) != 0)  
        goto fail;       // nunca é alcançada  
fail:  
    return err;           // 0 = "certificado válido"  
}
```

Dijkstra (1968)

"Go To Statement Considered Harmful"

Quem tem goto hoje

C, C++, C#, Go. Java não tem (p. 353)

Gotos restritos

break, continue e return melhoram a legibilidade (p. 353)

Com chaves obrigatórias (Go, Rust) ou com `gcc -Wall` (aviso de endentação enganosa), o bug teria sido pego.

Sebesta, 11ª ed. - §8.4 - p. 352-353 - exemplos/09_desvio_protegido

Esse slide fala do "goto", o comando que manda o programa pular direto para outro ponto do código. Ele volta ao bug da Apple de 2014: uma linha "goto fail;" ficou duplicada, então o programa sempre pulava para o fim antes de checar a assinatura do certificado, e devolvia "tudo certo" mesmo sem verificar nada.

O slide lembra que o Dijkstra, em 1968, já criticava o goto num artigo famoso, chamando-o de prejudicial. Hoje ele ainda existe em C, C++, C# e Go, mas o Java não tem. Versões mais restritas, como "break", "continue" e "return", ajudam a deixar o código mais legível.

No fim, mostra que esse bug teria sido evitado com chaves obrigatórias (como em Go e Rust) ou com um aviso do compilador sobre endentação enganosa.

Comandos protegidos de Dijkstra

```
if x >= y -> max := x
[] y >= x -> max := y
fi
```

Exemplo do livro (p. 354): se $x = y$, qualquer ramo serve.

- Todas as **guardas** são avaliadas
- Várias verdadeiras: escolha **não determinística**
- Nenhuma verdadeira: erro (no `if`) ou fim do laço (no `do ... od`)
- Base do CSP de Hoare (1978) e do `select` de Go

Go · `select`, herdeiro dos comandos protegidos

```
a <- "a"      // os dois canais
b <- "b"      // estão prontos

select {      // Go sorteia uma guarda
case x := <-a:
    contagem[x]++
case x := <-b:
    contagem[x]++
}

// 1000 repetições:
// map[a:505 b:495] (muda a cada execução)
```

Sebesta, 11ª ed. - §8.5 - p. 353-355 - exemplos/09_desvio_protegido

Esse slide fala dos comandos protegidos do Dijkstra, uma forma diferente de escolher caminhos. Cada opção tem uma condição (a "guarda"), e o programa avalia todas elas. Se só uma for verdadeira, ele segue por ela. Se várias forem, ele escolhe uma de forma não determinística, ou seja, qualquer uma serve e o resultado pode mudar a cada execução. No exemplo do livro, achar o maior entre dois valores: se forem iguais, tanto faz qual ramo rodar. Se nenhuma condição for verdadeira, dá erro (no "if") ou o laço simplesmente termina (no "do").

Essa ideia inspirou o modelo de comunicação entre processos de Hoare, de 1978, e o "select" do Go. Nele, quando dois canais estão prontos ao mesmo tempo, o Go sorteia um deles, e por isso, ao repetir mil vezes, os resultados ficam parecidos mas nunca iguais (505 para um, 495 para o outro).



1 · JavaScript

- Total: 0012
- for...in percorre as chaves (strings) e o + concatena
- Usar for...of ou reduce

2 · Python

- 180.0 (esperado 200.0)
- d or 10 trata o 0 como falso
- if d is None: d = 10

3 · Java

- inválido
- switch sem break cai do case 2 até o default
- Colocar break em cada caso ou usar switch com ->

4 · C

- saldo insuficiente e fim
- O ; depois do if é uma sentença vazia, então o printf fica fora do if
- Usar chaves sempre

5 · Go

- dentro: 20 e fora: 10
- := dentro do if cria uma nova variável x (sombreamento)
- Trocar por x = x * 2

6 · Java

- 0.0%
- 7 / 10 é divisão inteira (=0), feita antes do alargamento para double

c) $\text{acertos} * 100.0 / \text{total}$ ou $(\text{double}) \text{acertos} / \text{total} * 100$

Síntese: armadilhas e como as linguagens ajudam

Tema	Armadilha	Como as linguagens ajudam
Precedência	<code>-2 ** 2, 1 + 2 + "3"</code>	parênteses; JS obriga <code>(-2) ** 2</code>
Operandos	<code>a + f()</code> com efeito colateral (C, Go)	Java fixa a ordem; funções puras
Conversões	coerção silenciosa (C, JS, PHP)	Go e Rust exigem conversão explícita; <code>==</code>
Booleanos	<code>0, "", []</code> são falsos; <code>a > b > c</code> em C	if só com booleano (Java, Go, Rust); <code>??</code>
Atribuição	<code>if (x = y)</code>	atribuição não é expressão (Python, Go, Rust)
Seleção	switch sem break; else pendente	<code>-></code> , match exaustivo, chaves obrigatórias
Laços	<code>for...in</code> no JS; laço sem progresso	<code>for...of</code> , iteradores, break rotulado
Desvio	goto sem restrição	break, continue, return: gotos restritos

Sebesta, 11ª ed. - Caps. 7 e 8 - p. 299-361

Esse slide é o resumo da aula inteira: pra cada tema que você viu, ele mostra a armadilha mais comum e como as linguagens tentam proteger quem programa.

Na precedência, a confusão da ordem das contas é resolvida com parênteses, e o JavaScript até obriga a usá-los em certos casos. Nos operandos, o problema são funções que mexem em valores durante a conta, e o Java resolve fixando a ordem de leitura. Nas conversões, o perigo é a linguagem converter tipos em silêncio (C, JavaScript, PHP), e Go e Rust exigem que você peça tudo de forma explícita.

Nos booleanos, a pegadinha é que coisas como zero e texto vazio contam como falso, e Java, Go e Rust só aceitam um valor realmente booleano no "if". Na atribuição, o erro clássico é trocar "=" por "==" dentro do "if", e Python, Go e Rust evitam isso não tratando a atribuição como uma conta. Na seleção, os vilões são o "switch" sem "break" e o "else pendente", resolvidos com a seta do Java novo, o "match" que exige cobrir tudo e as chaves obrigatórias.

Nos laços, os problemas são o "for...in" do JavaScript e o laço que não progride, e a ajuda vem do "for...of", dos iteradores e do "break" rotulado. Por fim, no desvio, o "goto" sem restrição é substituído por versões controladas, como "break", "continue" e "return".

Em resumo: quase todo bug dessa aula vem de a linguagem fazer algo que parece diferente do que o código aparenta, e as linguagens modernas ajudam sendo mais explícitas e mais rígidas.

Atividades complementares (autopesquisa)

- Expressões em Ruby e em Lisp (§7.2.1.4-7.2.1.5, p. 305-306):
`expressoes.rb`, `expressoes.lisp`
- Expressões condicionais (§7.2.1.6, p. 306) e erros em expressões (§7.4.3, p. 313)
- Implementação da seleção múltipla: tabelas de desvio (§8.2.2.3, p. 337-339)
- Laços em linguagens funcionais: contagem por recursão (§8.3.1.4, p. 345)
- Iteradores e blocos de Ruby (§8.3.4, p. 349-352)
- Leitura: Dijkstra, "Go To Statement Considered Harmful" (1968)

Entrega sugerida

Um mesmo exemplo com `switch` em C, `switch ->` em Java, `switch` em Go e `match` em Rust: compare "cair" entre casos, exaustividade e legibilidade.

Próxima aula

Subprogramas (Caps. 9 e 10)

Sebesta, 11ª ed. - Caps. 7 e 8 - exemplos/11_autopesquisa

Esse slide fecha a aula com atividades de autopesquisa para quem quiser se aprofundar. Os temas são: como expressões funcionam em Ruby e Lisp, expressões condicionais e erros em expressões, como a seleção múltipla é implementada por baixo dos panos, laços em linguagens funcionais (que contam por recursão), iteradores e blocos do Ruby, e a leitura do artigo clássico do Dijkstra sobre o `goto`.

Há também uma entrega sugerida: escolher um mesmo exemplo e escrevê-lo com o "switch" do C, o "switch" com seta do Java, o "switch" do Go e o "match" do Rust, comparando como cada um lida com o "cair" entre casos, com a exigência de cobrir todas as possibilidades e com a legibilidade.

A próxima aula será sobre subprogramas (capítulos 9 e 10).